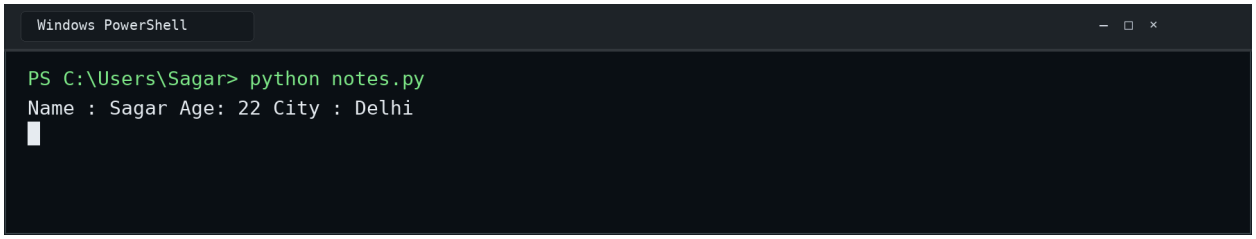
Python Programming Fundamentals

---------- Variable ----------

Explanation: Variables store values in memory and let us reuse those values by name.

```
name = "Sagar"
age = 22
city = "Delhi"
print(f"Name : {name} Age: {age} City : {city}")
```



#---------- DATA TYPES ----------

Explanation: Python provides built-in data types such as string, integer, float, boolean, list, tuple, set, and dictionary, each suited to different kinds of data.

```
name = "Sagar" #string
print(type(name))

age = 25 #integer
print(type(age))
```

```
height = 6.5 #float
print(type(height))

Married = False #bool
print(type(Married))

#list -> ordered , mutable
lst = [3,1,2]
print(type(lst))
lst[1] = 5
print(lst)

#1.append()
fruits = ["apple", "banana"]
fruits.append("cherry")
print(fruits)  # Output: ['apple', 'banana', 'cherry']

#2.insert()
fruits = ["apple", "cherry"]
fruits.insert(1, "banana")  # Index 1 par "banana" daal do
print(fruits)  # Output: ['apple', 'banana', 'cherry']

#3.remove()
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")  # Pehla "banana" hat jayega
print(fruits)  # Output: ['apple', 'cherry', 'banana']

#4.pop()
fruits = ["apple", "banana", "cherry"]
removed_item = fruits.pop(1)  # Index 1 ("banana") remove
```

```
print(removed_item)  # Output: banana
print(fruits)        # Output: ['apple', 'cherry']

#5.sort()
numbers = [4, 1, 3, 2]
numbers.sort()
print(numbers)  # Output: [1, 2, 3, 4]

#6.reverse()
nums = [1, 2, 3, 4]
nums.reverse()
print(nums)  # Output: [4, 3, 2, 1]

#7.count()
colors = ["red", "blue", "red", "green", "red"]
print(colors.count("red"))  # Output: 3

# 8.index()
animals = ["cat", "dog", "rabbit"]
print(animals.index("dog"))  # Output: 1



#tuple -> ordered,immutable
tple = (1,3,2)
print(type(tple))
print(tple[:-1])
# tple[1] = 5 #error

#1.count()
```

```
my_tuple = (1, 2, 3, 2, 4, 2)
count_two = my_tuple.count(2)
print(count_two)  # Output: 3

#2.index()
colors = ("red", "blue", "green", "blue")
pos = colors.index("blue")
print(pos)  # Output: 1

#3.len()
colors = ("red", "blue", "green", "blue")
print(len(colors))  # Output: 4


#set -> duplicate not allow , we can perform all set operations
st = {1,3,5}
st2 = {4,5,6}
print(type(st))
st.add(4)
st.discard(1)
print(st | st2)#union
print(st & st2)#intersection
print(st - st2)#difference
print(st^st2)#symmetric difference

# dictionaries -> key value data
dict = {
    "name":"Sagar",
    "age":22,
    "city":"Delhi"
```

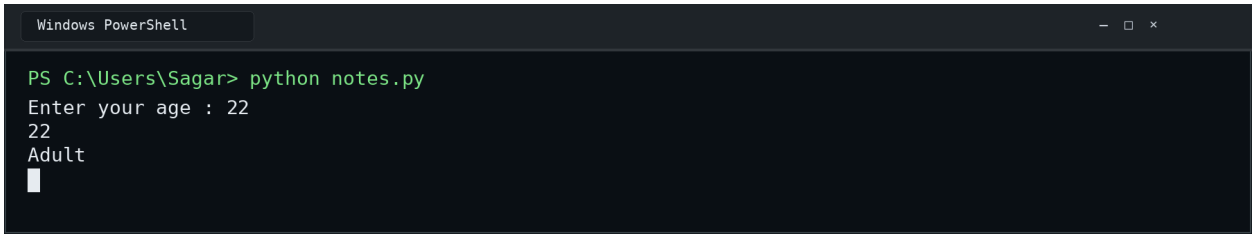
```
}
dict["course"] = "Python"
print(dict["name"])
print(dict["age"])
print(dict["course"])
print(dict)

# 1.get()
student = {"name": "Aman", "age": 21}
print(student.get("name"))     # Output: Aman
print(student.get("marks", 0)) # Output: 0

# 2.items()
student = {"name": "Aman", "age": 21}

for key, value in student.items():
    print(key, ":", value)
# Output:
# name : Aman
# age : 21

# 3.key() and values()
student = {"name": "Aman", "age": 21}
print(student.keys())    # Output: dict_keys(['name', 'age'])
print(student.values())  # Output: dict_values(['Aman', 21])

#4.update()
student = {"name": "Aman", "age": 21}
student.update({"age": 22, "city": "Delhi"})
print(student)
```

```
# Output: {'name': 'Aman', 'age': 22, 'city': 'Delhi'}

#5.pop()
student = {"name": "Aman", "age": 21, "city": "Delhi"}
removed_city = student.pop("city")
print(removed_city)  # Output: Delhi
print(student)       # Output: {'name': 'Aman', 'age': 21}
```

---------- CONDITIONAL STATEMENT ----------

Explanation: Conditional statements choose which block of code to execute based on a Boolean condition.

```
age = int(input("Enter your age : "))
print(age)
if age >= 20:
    print("Adult")
elif age >=13:
    print("Teenager")
else:
    print("Child")

status = "Adult" if age >= 18 else "Minor"  #shorthand conditional statement
#Comparision operators ->  == , != , > , < , >= and <=
#Logical operators -> and, or , not
```



#----------Loops----------

Explanation: Loops repeat a block of code efficiently; while repeats by condition, for iterates over a sequence or range, and break/continue control the loop flow.

1.While loop

```
i =1
while i<=5:
    print(i ,end=" ")
    i+=1
```

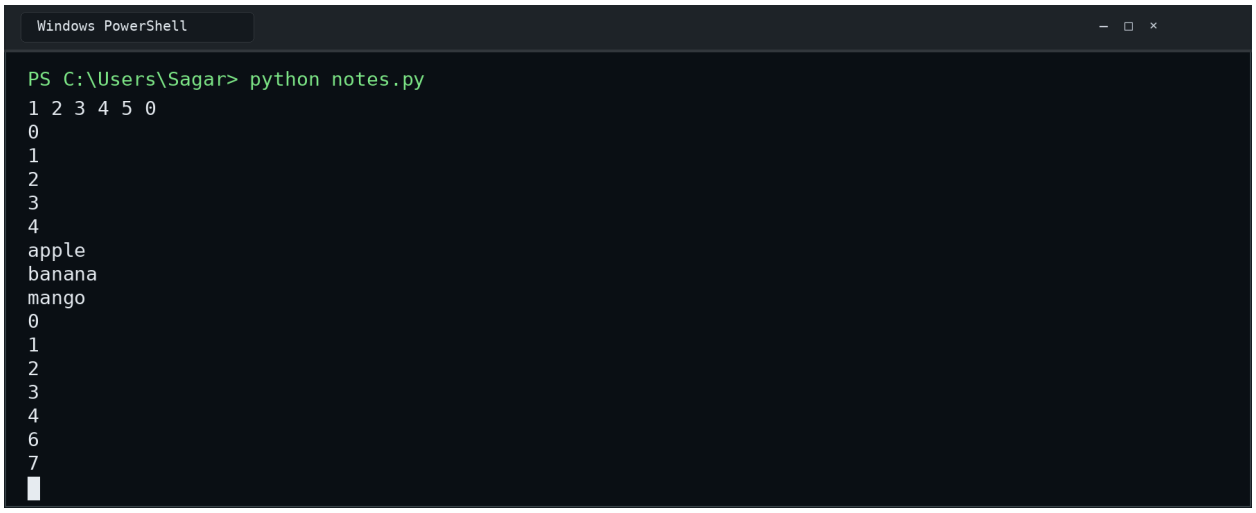
2.for loop

```
for i in range(5):
    print(i)
```

#with list

```
fruits = ["apple","banana","mango"]
for fruit in fruits:
    print(fruit)

for i in range(10):
    if i == 8:
        break
```

```
    if i == 5:
        continue
    print(i)
```

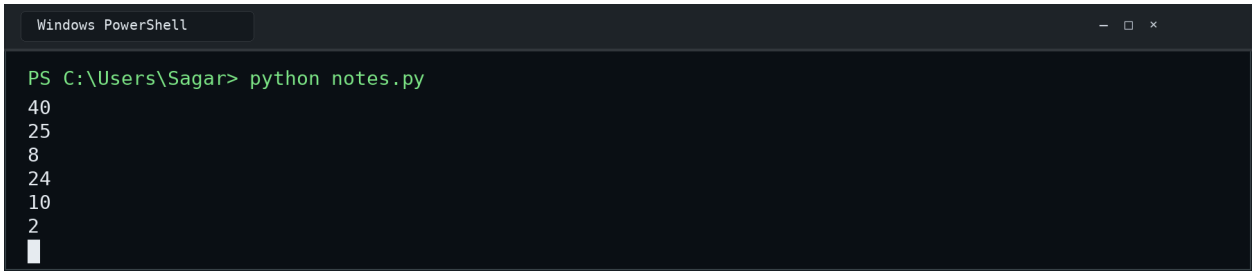


#----------Functions----------

Explanation: A function is a reusable block of code that can accept inputs, perform work, and optionally return a result.

```
def rectangle_area(length,width):
    area = length*width
    print(area)
rectangle_area(10,4)

def get_number():
    return 10

def add(a,b):
```

```
    return a+b
# built in functions -> print(),len(),type(),max(),min(),sum()

square = lambda x:x*x    #lambda function
print(square(5))
add = lambda a,b:a+b
print(add(5,3))

def factorial(n): # Recursive function
    if n == 0:
        return 1
    return n * factorial(n-1)

def genrFunc(a,b): # generator function
    yield a*b
    yield a+b
    yield a-b

for n in genrFunc(6,4):
    print(n)
```



#----------Parameter & Arguments----------

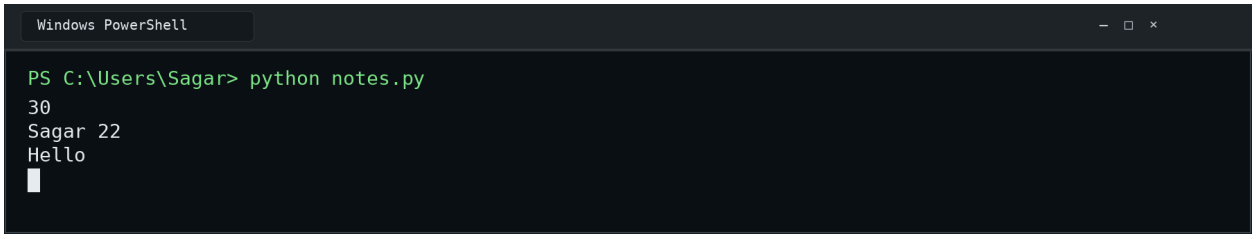
Explanation: **Parameters define what a function receives, while arguments are the actual values passed to it; Python supports positional, default, keyword, *args, and **kwargs forms.**

```
def student(name,age): #Positional parameter
    print(name,age)

def greet(name="User"): #Default parameters
    print("Hello ",name)

def student(name,age): #keyword arguments
    print(name,age)
student(age=21,name="Sagar")

def add(*num): #variable length arguments
    print(sum(num))
add(10,20)
add(10,20,30) # In argument values in the form of tuple

def student(**info): #Variable length keyword Arguments
    print(info)
student(name="Sagar",age=20,city="Delhi") #Arguments store in dictionary forms
```

#----------Return Values----------

Explanation: The return statement sends a value back to the caller, including multiple values packed into a tuple or even another function.

```
def add(a,b):
    return a+b
result = add(10,20)
print(result)

def get_user(): # multiple return
    return "Sagar",22 #it return value in tuple form
name,age = get_user()
print(name,age)

def outer(): # function return
    def inner():
        return "Hello"
    return inner
my_function = outer()
print(my_function())

def add(a:int,b:int)->int: # Return int and take argument only integer
    return a+b
```



#----------Exception Handling----------

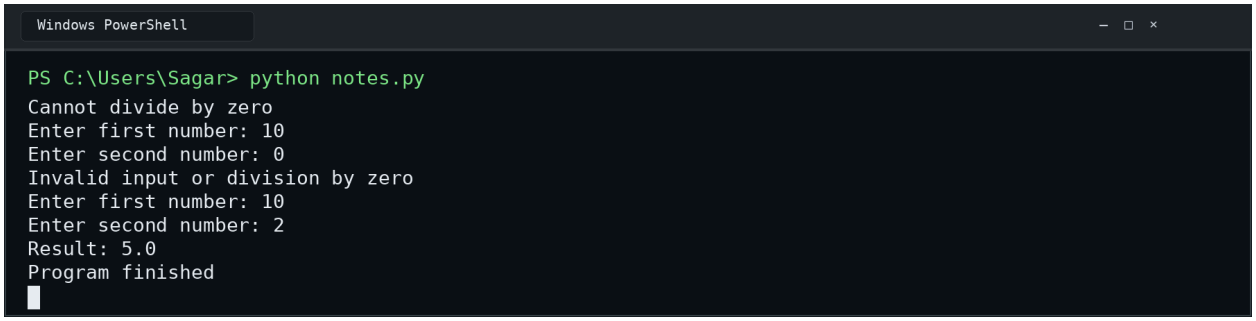
Explanation: Exception handling prevents a program from crashing unexpectedly by catching errors with try/except and optionally using else, finally, and raise.

```
try:
    a = 10
    b = 0
    print(a/b)
except ZeroDivisionError:
    print("Cannot divide by zero")
```

```
try: #multiple except blocks
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
    print(a/b)
except (ValueError,ZeroDivisionError):
    print("Invalid input or division by zero")
```

```
# all try + except + finally + else
try:
    a = int(input("Enter first number: "))
    b = int(input("Enter second number: "))
```

```
    result = a / b
except ValueError:
    print("Enter numbers only")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    print("Result:", result)
finally:
    print("Program finished")

# Raise your own exception
def withdraw(balance, amount):
    if amount > balance:
        raise ValueError("Insufficient balance")
    return balance - amount
```



#----------Module----------

Explanation:A module is a single Python file (.py) that contains reusable code such as variables, functions, classes, or statements.

Ex-

Suppose we create a file called `math_utils.py`:

 # math_utils.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b

PI = 3.14159
```

Now we can use this module from another Python file:

#another python file

```
import math_utils

print(math_utils.add(10, 20))
print(math_utils.subtract(20, 5))
print(math_utils.PI)
```

#----------Packages----------

Explanation: A package is a directory/folder that contains multiple Python modules and organizes them into a structured application.
Think of a package as a toolbox containing multiple smaller toolboxes.

Example of Package:-

```
myproject/
│
├── calculator/
│   ├── __init__.py
│   ├── addition.py
│   ├── subtraction.py
│   └── multiplication.py
```

```
│
└── main.py
```

Here:

- `calculator` → *Package*
- `addition.py` → *Module*
- `subtraction.py` → *Module*
- `multiplication.py` → *Module*

For example, `addition.py`:

```
def add(a, b):
    return a + b
```

main.py:

```
import calculator.addition import add
print(add(5,6))
```